EECS 348: Assignment 6

a. Name of two GenAI programs
ChatGPT and Claude

b. How you accessed them (e.g., Google query, CoPilot on my computer)
https://chatgpt.com/ and claude.ai

c. Prompt used. (You must use the same prompt for both GenAIs.)

I have a working C++ Sudoku solver written by another student. I need you to modify it, not rewrite it from scratch.

REQUIREMENTS

1. Keep it a single file (Assignment6.cpp), C++17, compiling with g++ on a Linux server.

2. Keep the existing recursive, object-oriented design (classes, backtracking). Keep existing class and method names where possible, and change only what is needed, so a diff against the original shows small targeted edits.

3. The program reads puzzle1.txt through puzzle5.txt and must write results for all five to a single file named solutions.txt.

4. For each puzzle, solutions.txt must contain:

- A line: Puzzle: x (x is the puzzle file name, e.g. puzzle1.txt)

- If no solution exists, one line: No solution found.

- Otherwise, EVERY solution (do not stop at the first), printed in the same format as the input puzzle files, followed by one line: Solutions found: n (n is the number of solutions)

5. All other existing functionality of the program must still work. If you find bugs in the original code, fix them.

6. Handle a missing or unreadable puzzle file without crashing.

7. Use efficient backtracking (for example, check row/column/box validity cheaply) and avoid unnecessary copying of the board.

8. Add a comment on every line or small block explaining what it does.

INPUT FORMAT EXAMPLE (puzzle1.txt):

5 _ _ _ _ _ 1 7 _

1 _ 6 5 _ 9 _ 4 _

4 7 2 1 _ 6 _ _ _

9 _ _ _ _ _ 5 _ _

_ 1 8 _ 9 5 4 _ _

6 _ _ 4 _ 2 3 8 9

_ 4 _ _ _ _ 9 3 _

_ 9 _ 7 _ 3 _ 5 _

2 6 3 9 5 8 7 1 4

ORIGINAL CODE:

```
/*
 * Program: EECS 348 Assignment 4 - Sudoku Solver
 * Description: Solves Sudoku puzzles with recursive, object-oriented
 *              backtracking and writes every solution to an output file.
 * Inputs: puzzle1.txt - puzzle5.txt (9 rows, digits or "_" for blank)
 * Outputs: solution1.txt - solution5.txt; each contains all solutions and
 *          a final solution count, or exactly "No solution found." if none.
 * Collaborators: ChatGPT and Gemini (source code generated by both tools);
 *                Claude (Anthropic) for review and guidance;
 * Other sources: ChatGPT, Gemini, Claude
 * Original author: Josselyn Bui
 * Creation date: 10/15/2026
 * Revised by: Dalton Brockert
 * Revision date: 10/29/2026
 *
```

```
 * Inherited code: SudokuSolver class structure, isValid(), findNextEmpty(),
 *                 recursive backtracking approach, and basic file loading.
 * Modified code: output-stream handling, strict input validation, output-file
 *                 generation, robust error handling, and solution formatting.
 */

#include <iostream>   // cout and cerr, used for status and error messages
#include <fstream>    // ifstream and ofstream, used for file input and output
#include <vector>     // holds the five input file names
#include <string>     // stores file names and input symbols

// ---------------------------------------------------------------------
// Block: SudokuSolver class
// Original class structure retained from Josselyn Bui's Assignment 4.
// Revisions by Dalton Brockert are marked in the comments below.
// ---------------------------------------------------------------------
class SudokuSolver {
private:
    int grid[9][9];       // the board; 0 means the cell is empty
    int solutionCount;    // how many complete solutions have been found

    // Inherited: checks whether num conflicts with a row, column, or block.
    bool isValid(int row, int col, int num) const {
        for (int c = 0; c < 9; ++c) {
```

```
        if (grid[row][c] == num) {
            return false;
        }
    }

    for (int r = 0; r < 9; ++r) {
        if (grid[r][col] == num) {
            return false;
        }
    }

    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;
    for (int r = 0; r < 3; ++r) {
        for (int c = 0; c < 3; ++c) {
            if (grid[startRow + r][startCol + c] == num) {
                return false;
            }
        }
    }

    return true;
}
```

```
// Inherited from the original program: checks that the original clues
// do not already contain duplicate digits in a row, column, or 3x3 block.
bool cluesConsistent() {
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            int digit = grid[r][c];
            if (digit == 0) {
                continue;
            }
            grid[r][c] = 0;                // temporarily lift the clue
            bool ok = isValid(r, c, digit);  // check against other clues
            grid[r][c] = digit;             // restore the clue
            if (!ok) {
                return false;
            }
        }
    }
    return true;
}

// Inherited: finds the first empty cell, scanning row by row.
bool findNextEmpty(int &row, int &col) const {
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
```

```
            if (grid[r][c] == 0) {
                row = r;
                col = c;
                return true;
            }
        }
    }
    return false;
}

// MODIFIED BY DALTON BROCKERT:
// Writes each completed grid directly to the selected output stream.
// No solution labels are written, so each grid retains input-file format.
void printGrid(std::ostream &out) const {
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            if (grid[r][c] == 0) {
                out << "_";
            } else {
                out << grid[r][c];
            }
            if (c < 8) {
                out << " ";
            }
```

```
        }
        out << "\n";
    }
}

// Inherited recursive backtracking, modified by Dalton Brockert to send
// every solution to the output file instead of stopping at the first.
void solveRecursive(std::ostream &out) {
    int row = -1;
    int col = -1;

    if (!findNextEmpty(row, col)) {
        ++solutionCount;
        printGrid(out);             // MODIFIED: solution goes to file
        return;                     // continue searching other branches
    }

    for (int val = 1; val <= 9; ++val) {
        if (isValid(row, col, val)) {
            grid[row][col] = val;
            solveRecursive(out);     // MODIFIED: pass output stream onward
            grid[row][col] = 0;      // backtrack and try next digit
        }
    }
```

```
    }

public:
    // Inherited constructor: initializes an empty board and zero solutions.
    SudokuSolver() : solutionCount(0) {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                grid[r][c] = 0;
            }
        }
    }

    // MODIFIED BY DALTON BROCKERT:
    // Reads exactly 81 valid tokens and rejects extra tokens, invalid symbols,
    // incomplete grids, or unreadable files. The board is reset before loading.
    bool loadFromFile(const std::string &filename) {
        std::ifstream inFile(filename.c_str());
        if (!inFile.is_open()) {
            return false;
        }

        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                grid[r][c] = 0;
```

```
    }
}
solutionCount = 0;

for (int r = 0; r < 9; ++r) {
    for (int c = 0; c < 9; ++c) {
        std::string token;
        if (!(inFile >> token)) {
            return false;        // fewer than 81 cells
        }

        if (token == "_") {
            grid[r][c] = 0;
        } else if (token.size() == 1 &&
                token[0] >= '1' && token[0] <= '9') {
            grid[r][c] = token[0] - '0';
        } else {
            return false;        // invalid cell token
        }
    }
}

// MODIFIED: reject trailing tokens; valid input has exactly 81 cells.
std::string extraToken;
```

```
    if (inFile >> extraToken) {
        return false;
    }
    if (inFile.bad()) {
        return false;               // read error
    }

    return true;
}
    // Prints the original puzzle to the console.
void printGrid() const {
    printGrid(std::cout);
}

// Finds every solution and writes the results to the output file.
bool solveToFile(std::ostream &out) {
    solutionCount = 0;

    if (!cluesConsistent()) {
        out << "No solution found.\n";
        return static_cast<bool>(out);
    }

    solveRecursive(out);
```

```
        if (solutionCount == 0) {
            out << "No solution found.\n";
        } else {
            out << "Solutions found: " << solutionCount << "\n";
        }

        return static_cast<bool>(out);
    }

    // Original compatibility method.
    void solve() {
        solveToFile(std::cout);
    }
};

// Main function processes all five Sudoku puzzles.
int main() {
    const std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
```

```
};

for (std::size_t i = 0; i < puzzleFiles.size(); ++i) {
    const std::string &filename = puzzleFiles[i];

    const std::string outputFilename =
        "solution" + std::string(1, static_cast<char>('1' + i)) + ".txt";

    std::cout << "========================================\n";
    std::cout << filename << " -> " << outputFilename << "\n";
    std::cout << "========================================\n";

    std::ofstream outFile(outputFilename.c_str());

    if (!outFile.is_open()) {
        std::cerr << "Error: Could not create "
                  << outputFilename << "\n";
        continue;
    }

    SudokuSolver solver;

    if (!solver.loadFromFile(filename)) {
        std::cerr << "Error: Missing, unreadable, or malformed input: "
```

```
            << filename << "\n";

    outFile << "No solution found.\n";
    outFile.close();
    continue;
}

std::cout << "Original Puzzle:\n";
solver.printGrid();

std::cout << "\nSolving; results are being written to "
          << outputFilename << "...\n";

if (!solver.solveToFile(outFile)) {
    std::cerr << "Error: Failed while writing "
              << outputFilename << "\n";
}

outFile.close();

if (outFile.fail()) {
    std::cerr << "Error: Failed to finish writing "
              << outputFilename << "\n";
}
```

```
        std::cout << "\n";
    }


    return 0;
}
```

Return the complete modified Assignment6.cpp in one code block. After the code, briefly list what you changed and why.

d. Generated code from both GenAIs.
ChatGPT Code:

```
/*
 * Program: EECS 348 Assignment 6 - Sudoku Solver
 * Description: Solves Sudoku puzzles using recursive, object-oriented
 *              backtracking and writes every solution to one output file.
 * Inputs: puzzle1.txt - puzzle5.txt (9 rows, digits or "_" for blank)
 * Output: solutions.txt (all solutions for all five puzzles)
 *
 * Original author: Josselyn Bui
 * Revised by: Dalton Brockert
 *
 * Inherited code: SudokuSolver class structure, isValid(),
 *                 findNextEmpty(), and recursive backtracking.
 * Modified code: Efficient validity checking, solution counting,
 *                 combined output file, and robust file handling.
 */

#include <iostream>   // Provides cout and cerr for console output.
#include <fstream>    // Provides ifstream and ofstream for file operations.
#include <vector>     // Stores the five input puzzle filenames.
#include <string>     // Stores filenames and input tokens.
#include <cstdint>    // Provides fixed-width unsigned integer types.

// ---------------------------------------------------------------------
// SudokuSolver class: Maintains the board and solves Sudoku puzzles.
// ---------------------------------------------------------------------
class SudokuSolver {
private:
```

```
int grid[9][9];             // Stores the Sudoku board; 0 represents empty.
int solutionCount;          // Tracks the total number of solutions.

// Bit masks track digits already used in each row, column, and box.
// Bit positions 1 through 9 represent the corresponding Sudoku digits.
std::uint16_t rowUsed[9];
std::uint16_t colUsed[9];
std::uint16_t boxUsed[9];

// Returns the index of the 3x3 box containing a given cell.
int boxIndex(int row, int col) const {
    return (row / 3) * 3 + (col / 3);
}

// Checks whether num is available in the specified row, column, and box.
bool isValid(int row, int col, int num) const {
    // Create a bit mask representing the digit being checked.
    std::uint16_t bit = static_cast<std::uint16_t>(1u << num);

    // A digit is invalid if its bit is already set in any constraint.
    return !(rowUsed[row] & bit) &&
           !(colUsed[col] & bit) &&
           !(boxUsed[boxIndex(row, col)] & bit);
}

// Places a digit on the board and updates the corresponding bit masks.
void placeNumber(int row, int col, int num) {
    // Calculate the bit representing the selected digit.
    std::uint16_t bit = static_cast<std::uint16_t>(1u << num);

    // Store the digit in the board.
    grid[row][col] = num;

    // Mark the digit as used in its row, column, and box.
    rowUsed[row] |= bit;
    colUsed[col] |= bit;
    boxUsed[boxIndex(row, col)] |= bit;
}

// Removes a digit during backtracking and updates the bit masks.
void removeNumber(int row, int col, int num) {
    // Calculate the bit representing the digit being removed.
    std::uint16_t bit = static_cast<std::uint16_t>(1u << num);
```

```
    // Clear the digit from the board.
    grid[row][col] = 0;

    // Remove the digit from its row, column, and box masks.
    rowUsed[row] &= static_cast<std::uint16_t>(~bit);
    colUsed[col] &= static_cast<std::uint16_t>(~bit);
    boxUsed[boxIndex(row, col)] &= static_cast<std::uint16_t>(~bit);
}

// Checks that the original puzzle contains no duplicate clues.
bool cluesConsistent() {
    // Reset all constraint masks before checking the original clues.
    for (int i = 0; i < 9; ++i) {
        rowUsed[i] = 0;
        colUsed[i] = 0;
        boxUsed[i] = 0;
    }

    // Examine every cell in the original puzzle.
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            // Read the current cell's value.
            int digit = grid[r][c];

            // Empty cells do not need consistency checking.
            if (digit == 0) {
                continue;
            }

            // Reject digits that already appear in the same constraints.
            if (!isValid(r, c, digit)) {
                return false;
            }

            // Record this clue in the row, column, and box masks.
            std::uint16_t bit =
                static_cast<std::uint16_t>(1u << digit);

            rowUsed[r] |= bit;
            colUsed[c] |= bit;
            boxUsed[boxIndex(r, c)] |= bit;
        }
    }
```

```
    // All original clues are consistent.
    return true;
}

// Finds the next empty cell, scanning from left to right and top to bottom.
bool findNextEmpty(int &row, int &col) const {
    // Examine every row.
    for (int r = 0; r < 9; ++r) {
        // Examine every column in the current row.
        for (int c = 0; c < 9; ++c) {
            // Return the first empty cell found.
            if (grid[r][c] == 0) {
                row = r;
                col = c;
                return true;
            }
        }
    }

    // No empty cells remain.
    return false;
}

// Prints the completed Sudoku board in the original input format.
void printGrid(std::ostream &out) const {
    // Print all nine rows.
    for (int r = 0; r < 9; ++r) {
        // Print all nine columns in the current row.
        for (int c = 0; c < 9; ++c) {
            // Print an underscore for an empty cell or its digit otherwise.
            if (grid[r][c] == 0) {
                out << "_";
            } else {
                out << grid[r][c];
            }

            // Separate cell values with spaces.
            if (c < 8) {
                out << " ";
            }
        }

        // End the current row.
        out << "\n";
```

```
        }
    }

    // Recursively searches for every possible solution using backtracking.
    void solveRecursive(std::ostream &out) {
        // Initialize the coordinates of the next empty cell.
        int row = -1;
        int col = -1;

        // If there are no empty cells, a complete solution has been found.
        if (!findNextEmpty(row, col)) {
            // Increase the number of solutions discovered.
            ++solutionCount;

            // Write this complete solution directly to the output file.
            printGrid(out);

            // Return to the previous recursive call to continue searching.
            return;
        }

        // Try each possible Sudoku digit from 1 through 9.
        for (int val = 1; val <= 9; ++val) {
            // Only try digits that do not conflict with existing constraints.
            if (isValid(row, col, val)) {
                // Place the candidate digit and update the constraint masks.
                placeNumber(row, col, val);

                // Recursively search for additional solutions.
                solveRecursive(out);

                // Remove the candidate digit to restore the previous board.
                removeNumber(row, col, val);
            }
        }
    }

public:
    // Constructor initializes an empty board and resets all tracking data.
    SudokuSolver() : solutionCount(0) {
        // Initialize all constraint masks to indicate no used digits.
        for (int i = 0; i < 9; ++i) {
            rowUsed[i] = 0;
            colUsed[i] = 0;
```

```
        boxUsed[i] = 0;
    }

    // Initialize every board cell to zero.
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            grid[r][c] = 0;
        }
    }
}

// Reads and validates exactly 81 cells from the specified puzzle file.
bool loadFromFile(const std::string &filename) {
    // Open the requested input file.
    std::ifstream inFile(filename.c_str());

    // Return false if the file is missing or cannot be opened.
    if (!inFile.is_open()) {
        return false;
    }

    // Reset the board before loading a new puzzle.
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            grid[r][c] = 0;
        }
    }

    // Reset the solution counter.
    solutionCount = 0;

    // Reset all row, column, and box tracking masks.
    for (int i = 0; i < 9; ++i) {
        rowUsed[i] = 0;
        colUsed[i] = 0;
        boxUsed[i] = 0;
    }

    // Read exactly nine rows and nine columns.
    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            // Read the next whitespace-separated cell token.
            std::string token;
```

```
            // Reject files containing fewer than 81 cells.
            if (!(inFile >> token)) {
                return false;
            }

            // Store zero for an underscore representing an empty cell.
            if (token == "_") {
                grid[r][c] = 0;
            }
            // Accept only single-digit values from 1 through 9.
            else if (token.size() == 1 &&
                     token[0] >= '1' && token[0] <= '9') {
                grid[r][c] = token[0] - '0';
            }
            // Reject invalid cell tokens.
            else {
                return false;
            }
        }
    }

    // Reject any additional tokens after the required 81 cells.
    std::string extraToken;
    if (inFile >> extraToken) {
        return false;
    }

    // Reject input errors that occurred while reading the file.
    if (inFile.bad()) {
        return false;
    }

    // The puzzle has exactly 81 valid cell tokens.
    return true;
}

// Prints the original puzzle to the console for compatibility.
void printGrid() const {
    // Reuse the existing output-stream version of printGrid().
    printGrid(std::cout);
}

// Solves the puzzle and writes every solution to the supplied stream.
bool solveToFile(std::ostream &out) {
```

```
        // Reset the solution counter before starting the search.
        solutionCount = 0;

        // Reject puzzles containing conflicting original clues.
        if (!cluesConsistent()) {
            // Report that no valid solution exists.
            out << "No solution found.\n";

            // Return whether the output operation succeeded.
            return static_cast<bool>(out);
        }

        // Begin recursive backtracking to find every possible solution.
        solveRecursive(out);

        // Report when the search did not discover any solutions.
        if (solutionCount == 0) {
            out << "No solution found.\n";
        }
        // Otherwise, report the total number of solutions discovered.
        else {
            out << "Solutions found: " << solutionCount << "\n";
        }

        // Return whether all output operations succeeded.
        return static_cast<bool>(out);
    }

    // Original compatibility method: solves the puzzle and prints to console.
    void solve() {
        // Reuse the existing solver with standard output.
        solveToFile(std::cout);
    }
};

// ----------------------------------------------------------------------
// Main function: Processes all five puzzle files into solutions.txt.
// ----------------------------------------------------------------------
int main() {
    // Store the five required input filenames.
    const std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
```

```
    "puzzle4.txt",
    "puzzle5.txt"
};

// Open one shared output file for all five puzzles.
std::ofstream outFile("solutions.txt");

// Stop safely if the output file cannot be created or opened.
if (!outFile.is_open()) {
    std::cerr << "Error: Could not create solutions.txt\n";
    return 1;
}

// Process each puzzle in the required order.
for (std::size_t i = 0; i < puzzleFiles.size(); ++i) {
    // Get the filename of the current puzzle.
    const std::string &filename = puzzleFiles[i];

    // Display a separator and the current puzzle filename.
    std::cout << "========================================\n";
    std::cout << "Processing: " << filename << "\n";
    std::cout << "========================================\n";

    // Write the required puzzle header to the shared output file.
    outFile << "Puzzle: " << filename << "\n";

    // Create a separate solver object for the current puzzle.
    SudokuSolver solver;

    // Attempt to load the current input puzzle.
    if (!solver.loadFromFile(filename)) {
        // Display an error without terminating the entire program.
        std::cerr << "Error: Missing, unreadable, or malformed input: "
                  << filename << "\n";

        // Write the required no-solution message for this puzzle.
        outFile << "No solution found.\n";

        // Separate this puzzle's output from the next puzzle.
        outFile << "\n";

        // Continue processing the remaining puzzle files.
        continue;
    }
```

```
        // Display the original puzzle in the console.
        std::cout << "Original Puzzle:\n";
        solver.printGrid();

        // Inform the user that all results are going to solutions.txt.
        std::cout << "\nSolving; results are being written to solutions.txt...\n";

        // Solve the puzzle and write every solution to the shared file.
        if (!solver.solveToFile(outFile)) {
            // Report output errors without crashing.
            std::cerr << "Error: Failed while writing solutions.txt\n";
        }

        // Add a blank line to separate consecutive puzzle results.
        outFile << "\n";

        // Report whether the output stream is still functioning.
        if (!outFile) {
            std::cerr << "Error: Output file writing failed.\n";
            break;
        }

        // Display a blank line between puzzles in the console.
        std::cout << "\n";
    }

    // Close the shared output file after all puzzles have been processed.
    outFile.close();

    // Check whether closing or flushing the file produced an error.
    if (outFile.fail()) {
        std::cerr << "Error: Failed to finish writing solutions.txt\n";
        return 1;
    }

    // Report successful completion.
    std::cout << "All puzzle processing completed.\n";
    std::cout << "Results saved in solutions.txt\n";

    // Return zero to indicate successful program execution.
    return 0;
}
```

Claude Code:

```
/*
 * Program: EECS 348 Assignment 6 - Sudoku Solver                    // MODIFIED: was
Assignment 4
 * Description: Solves Sudoku puzzles with recursive, object-oriented
 *              backtracking and writes every solution to one output file.
 * Inputs: puzzle1.txt - puzzle5.txt (9 rows, digits or "_" for blank)
 * Outputs: solutions.txt; for each puzzle it contains "Puzzle: <file>",
 *          then all solutions plus "Solutions found: n", or exactly
 *          "No solution found." if there are none.                  // MODIFIED: single output file
 * Collaborators: ChatGPT and Gemini (source code generated by both tools);
 *                Claude (Anthropic) for review, guidance, and the
 *                Assignment 6 modifications;                       // MODIFIED
 * Other sources: ChatGPT, Gemini, Claude
 * Original author: Josselyn Bui
 * Creation date: 10/15/2026
 * Revised by: Dalton Brockert
 * Revision date: 10/29/2026
 * Revised by: [YOUR FULL NAME]                                     // MODIFIED: add your name
 * Revision date: [TODAY'S DATE]                                    // MODIFIED: add your date
 *
 * Inherited code: SudokuSolver class structure, findNextEmpty(),
 *                 recursive backtracking approach, and basic file loading.
 * Modified code: single solutions.txt output with "Puzzle: x" headers,
 *                bitmask-based validity checks, start-index cell search.  // MODIFIED
 */

#include <iostream>   // cout and cerr, used for status and error messages
#include <fstream>    // ifstream and ofstream, used for file input and output
#include <vector>     // holds the five input file names
#include <string>     // stores file names and input symbols
#include <cstddef>    // MODIFIED: std::size_t for the loop index in main

// ---------------------------------------------------------------------
// Block: SudokuSolver class
// Original class structure retained from Josselyn Bui's Assignment 4.
// Revisions are marked "MODIFIED" in the comments below.
// ---------------------------------------------------------------------
class SudokuSolver {
private:
    int grid[9][9];       // the board; 0 means the cell is empty
    int solutionCount;    // how many complete solutions have been found
    int rowMask[9];       // MODIFIED: bit d is set if digit d is used in that row
    int colMask[9];       // MODIFIED: bit d is set if digit d is used in that column
```

```
int boxMask[9];       // MODIFIED: bit d is set if digit d is used in that 3x3 box

// MODIFIED: resets all three bitmask arrays to "no digits used".
void clearMasks() {
    for (int i = 0; i < 9; ++i) {   // visit each row, column, and box slot
        rowMask[i] = 0;             // no digits used in this row
        colMask[i] = 0;             // no digits used in this column
        boxMask[i] = 0;             // no digits used in this box
    }
}

// MODIFIED: O(1) validity check using the bitmasks instead of scanning
// 27 cells. Box index is (row / 3) * 3 + (col / 3).
bool isValid(int row, int col, int num) const {
    int used = rowMask[row] | colMask[col]
             | boxMask[(row / 3) * 3 + (col / 3)];  // all digits blocked for this cell
    return (used & (1 << num)) == 0;                // valid if bit for num is clear
}

// MODIFIED: puts a digit on the board and records it in the bitmasks.
void placeDigit(int row, int col, int num) {
    int bit = 1 << num;                     // bit that represents this digit
    grid[row][col] = num;                   // write the digit into the board
    rowMask[row] |= bit;                    // mark digit used in the row
    colMask[col] |= bit;                    // mark digit used in the column
    boxMask[(row / 3) * 3 + (col / 3)] |= bit;  // mark digit used in the box
}

// MODIFIED: removes the digit from a cell and clears it from the bitmasks.
void removeDigit(int row, int col) {
    int bit = 1 << grid[row][col];          // bit of the digit being removed
    grid[row][col] = 0;                     // empty the cell
    rowMask[row] &= ~bit;                   // digit no longer used in the row
    colMask[col] &= ~bit;                   // digit no longer used in the column
    boxMask[(row / 3) * 3 + (col / 3)] &= ~bit;  // digit no longer used in the box
}

// MODIFIED: checks the original clues for duplicates AND builds the
// bitmasks from them. Returns false if two clues conflict.
bool cluesConsistent() {
    clearMasks();                           // start from a clean mask state
    for (int r = 0; r < 9; ++r) {           // go through every row
        for (int c = 0; c < 9; ++c) {       // go through every column
            int digit = grid[r][c];         // clue (or 0 if blank)
```

```
            if (digit == 0) {               // blank cells have nothing to check
                continue;
            }
            if (!isValid(r, c, digit)) {     // digit already used in row/col/box
                return false;                // duplicate clue means no solution
            }
            placeDigit(r, c, digit);         // record the clue in the masks
        }
    }
    return true;                             // all clues are consistent
}

// Inherited: finds the first empty cell, scanning row by row.
// MODIFIED: starts scanning at "start" (0 to 80) instead of cell 0,
// because every cell before the last placed one is already filled.
bool findNextEmpty(int start, int &row, int &col) const {
    for (int i = start; i < 81; ++i) {       // MODIFIED: single index, row-major order
        int r = i / 9;                       // row of cell i
        int c = i % 9;                       // column of cell i
        if (grid[r][c] == 0) {               // first empty cell found
            row = r;                         // report its row
            col = c;                         // report its column
            return true;
        }
    }
    return false;                            // board is full
}

// Writes each completed grid directly to the selected output stream.
// No solution labels are written, so each grid retains input-file format.
void printGrid(std::ostream &out) const {
    for (int r = 0; r < 9; ++r) {            // each row
        for (int c = 0; c < 9; ++c) {        // each column
            if (grid[r][c] == 0) {
                out << "_";                  // blank cell
            } else {
                out << grid[r][c];           // digit
            }
            if (c < 8) {
                out << " ";                  // space between cells
            }
        }
        out << "\n";                         // end of row
    }
```

```
    }

    // Inherited recursive backtracking that outputs every solution.
    // MODIFIED: takes a start index, and uses placeDigit/removeDigit so the
    // bitmasks stay in sync with the grid.
    void solveRecursive(std::ostream &out, int start) {
        int row = -1;                       // row of the next empty cell
        int col = -1;                       // column of the next empty cell

        if (!findNextEmpty(start, row, col)) {   // MODIFIED: pass start index
            ++solutionCount;                    // board is full: one more solution
            printGrid(out);                     // write it to the file
            return;                             // continue searching other branches
        }

        for (int val = 1; val <= 9; ++val) {     // try digits in ascending order
            if (isValid(row, col, val)) {        // skip digits that conflict
                placeDigit(row, col, val);       // MODIFIED: place and update masks
                solveRecursive(out, row * 9 + col + 1);  // MODIFIED: resume after this cell
                removeDigit(row, col);           // MODIFIED: backtrack and update masks
            }
        }
    }

public:
    // Inherited constructor: initializes an empty board and zero solutions.
    SudokuSolver() : solutionCount(0) {
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                grid[r][c] = 0;                 // every cell starts empty
            }
        }
        clearMasks();                           // MODIFIED: no digits used yet
    }

    // Reads exactly 81 valid tokens and rejects extra tokens, invalid symbols,
    // incomplete grids, or unreadable files. The board is reset before loading.
    bool loadFromFile(const std::string &filename) {
        std::ifstream inFile(filename.c_str());  // open the puzzle file
        if (!inFile.is_open()) {
            return false;                       // missing or unreadable file
        }

        for (int r = 0; r < 9; ++r) {
```

```
        for (int c = 0; c < 9; ++c) {
            grid[r][c] = 0;                 // reset the board
        }
    }
    clearMasks();                           // MODIFIED: reset masks with the board
    solutionCount = 0;                      // reset the solution counter

    for (int r = 0; r < 9; ++r) {
        for (int c = 0; c < 9; ++c) {
            std::string token;              // one cell symbol
            if (!(inFile >> token)) {
                return false;               // fewer than 81 cells
            }

            if (token == "_") {
                grid[r][c] = 0;             // blank cell
            } else if (token.size() == 1 &&
                    token[0] >= '1' && token[0] <= '9') {
                grid[r][c] = token[0] - '0'; // clue digit
            } else {
                return false;               // invalid cell token
            }
        }
    }

    std::string extraToken;                 // reject trailing tokens
    if (inFile >> extraToken) {
        return false;                       // more than 81 cells
    }
    if (inFile.bad()) {
        return false;                       // read error
    }

    return true;                            // puzzle loaded correctly
}

// Prints the original puzzle to the console.
void printGrid() const {
    printGrid(std::cout);
}

// Finds every solution and writes the results to the output stream.
bool solveToFile(std::ostream &out) {
    solutionCount = 0;                      // start counting from zero
```

```
        if (!cluesConsistent()) {               // conflicting clues: no solution
            out << "No solution found.\n";
            return static_cast<bool>(out);
        }

        solveRecursive(out, 0);                 // MODIFIED: begin at cell 0

        if (solutionCount == 0) {
            out << "No solution found.\n";      // search found nothing
        } else {
            out << "Solutions found: " << solutionCount << "\n";  // final count
        }

        return static_cast<bool>(out);          // false if writing failed
    }

    // Original compatibility method.
    void solve() {
        solveToFile(std::cout);
    }
};

// Main function processes all five Sudoku puzzles.
int main() {
    const std::vector<std::string> puzzleFiles = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // MODIFIED: open the single output file once, before the loop.
    std::ofstream outFile("solutions.txt");
    if (!outFile.is_open()) {                   // cannot continue without it
        std::cerr << "Error: Could not create solutions.txt\n";
        return 1;
    }

    for (std::size_t i = 0; i < puzzleFiles.size(); ++i) {
        const std::string &filename = puzzleFiles[i];   // current puzzle file

        std::cout << "========================================\n";
```

```
        std::cout << filename << " -> solutions.txt\n";  // MODIFIED: shared output file
        std::cout << "========================================\n";

        outFile << "Puzzle: " << filename << "\n";       // MODIFIED: required header line

        SudokuSolver solver;                    // fresh solver for each puzzle

        if (!solver.loadFromFile(filename)) {
            std::cerr << "Error: Missing, unreadable, or malformed input: "
                      << filename << "\n";

            outFile << "No solution found.\n";   // report and move to the next puzzle
            continue;                           // MODIFIED: do not close the shared file
        }

        std::cout << "Original Puzzle:\n";
        solver.printGrid();                     // show the clues on the console

        std::cout << "\nSolving; results are being written to "
                  << "solutions.txt...\n";       // MODIFIED: shared output file

        if (!solver.solveToFile(outFile)) {      // search and write all solutions
            std::cerr << "Error: Failed while writing solutions.txt\n";
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness and execution time. Both programs read the five puzzle files, write a Puzzle: x header for each, print every solution in the input format, and end with Solutions found: n or No solution found.. Both also reject duplicate clues, malformed grids, and missing files without crashing. The only visible output difference is that the ChatGPT version adds a blank line after each puzzle, which the assignment does not ask for. For execution time, both use bitmasks for rows, columns, and boxes, so each validity check is O(1) instead of a scan of 27 cells. Both are still O(9^k) in the worst case, where k is the number of blank cells, because neither picks the most constrained cell first. The difference is in finding the next blank cell. The ChatGPT version rescans the board from the first cell on every recursive call, up to 81 checks per node. The Claude version resumes from the cell after the one just filled, so the scan is usually short. In my testing on the Cycle server, the Claude version performed better, which is consistent with this lower per-node cost.

Space complexity and maintainability. Space is nearly identical. Both store a 9x9 integer board (324 bytes) and three arrays of nine bitmasks, which total 108 bytes in the Claude version (int) and 54 bytes in the ChatGPT version (uint16_t). Recursion depth is at most the number of blank cells (at most 81), and solutions are written straight to the file instead of being stored, so extra

space is O(k) in both. For maintainability, the ChatGPT version has a boxIndex() helper and a comment on nearly every line, but it copy-pastes the mask reset in three places and re-implements mask updating inside cluesConsistent. It also dropped the original prolog fields (creation date, collaborators, other sources) and the earlier authorship tags. The Claude version resets masks in a single clearMasks() function, reuses placeDigit and removeDigit, keeps the prolog, and marks each changed line with a MODIFIED tag, which keeps the changes small and the authorship clear. Its weaknesses are that the box-index formula is repeated three times and the magic numbers 9, 3, and 81 appear throughout.

f. Select one to use as the basis for this assignment and justify why you picked it.

I selected the Claude version as the base for this assignment. In my testing on the Cycle server it ran better than the ChatGPT version, and the code explains why: it resumes searching for the next blank cell from the last filled cell instead of rescanning the whole board on every recursive call, which lowers the work done at each node. Both versions are equally correct in format and use the same bitmask validity check, so they are close on correctness and space (the 54-byte difference in mask storage is negligible), and execution time was the clearest performance difference. Claude's version is also easier to maintain and to build on. It resets the bitmasks in one clearMasks() function instead of three copied loops, and it reuses placeDigit and removeDigit instead of repeating logic. It also keeps the prolog fields and the authorship tags from earlier contributors, and it marks every changed line with a MODIFIED comment. That keeps the changes small and visible in the GitHub history, which this assignment requires (the code must be modified, not overwritten) and which supports accurate credit for collaborators. For these reasons, Claude's version was the stronger starting point.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness and execution time. I started from the Claude version and made three changes. For correctness, I changed solutionCount from int to long long, so a puzzle with a very large number of solutions cannot overflow the counter and print a wrong Solutions found: n. Because the program tries every solution, that count can grow large. The output format and order are unchanged, since cells are still filled in row-major order and digits are still tried in ascending order. The base case also still works for a puzzle with no blank cells (the blank list is empty, so it reports exactly one solution). For execution time, I replaced the per-call scan for the next empty cell with a list of blank cells, built once by buildBlankList() after the clues pass the consistency check. The recursion now advances by index into that list instead of scanning the board on every call. I also replaced the loop over digits 1 to 9 (which called isValid on each one) with an available bitmask built from the row, column, and box masks. The loop only visits legal digits, using available & -available to take the smallest one and available &= available - 1 to clear it. This lowers the work done at each node of the search, since no scanning is left and fewer failed digit checks are made. The worst case is still O(9^k) for k blank cells, because cells are not chosen by how constrained they are, so the improvement is in the cost per node rather than the growth rate.

Space complexity and maintainability. Space stays essentially the same. The blank-cell list adds at most 81 bytes (an array of unsigned char) plus one integer, which is negligible next to the 324-byte board. Recursion depth is unchanged at no more than the number of blank cells, and solutions are still written straight to the file instead of being stored, so extra space remains O(k). For maintainability, I removed findNextEmpty() and its start-index parameter, since the blank list makes them unnecessary, which leaves one less function to maintain and a simpler recursive signature. I tagged every block I added with MODIFIED BY JOSSELYN FOR ASSIGNMENT 6, so the changes are easy to find and the authorship is clear, and I commented the new logic line by line in my own words. There is one tradeoff: the bit operations and __builtin_ctz (a GCC feature, which works on the Cycle servers) are harder to read than a plain loop from 1 to 9, so I explained each bit operation in comments to offset that.